

C# (.NET)

Windows Form

Parte de .Net utilizado para crear aplicaciones para clientes Windows

Los formularios son ventanas que interactúan con el usuario, los controles son los objetos que colocamos dentro de dichas ventanas, un formulario por tanto tiene como misión principal la de contenedor de controles. A estos formularios en el entorno .NET se les llama WinForms para distinguirlos de los WebForms (formularios ASP).

Namespace System.Windows.Forms

Es el espacio de nombres base para este tipo de aplicaciones, en él están los controles que usaremos. Este namespace usa frecuentemente el System.Drawing que contiene información acerca de colores, fuentes,

Formularios : Propiedades, Métodos y Eventos

Típicas ventanas Windows que sirven sobre todo como contenedores de otros controles.

Propiedades

PROPERTY	DESCRIPTION
(Name) This	Sets the name of the <i>Form</i> class shown in the Designer. property can be set only at design time.
<i>BackColor</i>	Indicates the background color of the form.
<i>BackgroundImage</i>	Indicates the background image of the form.
<i>BackgroundImageLayout</i>	Determines how the image indicated by the <i>Background-Image</i> property will be laid out on the form. If no background image is selected, this property has no effect.
<i>ControlBox</i> menu	Determines whether the form has a Control/System box.

<i>Cursor</i> moved	Indicates the cursor that appears when the cursor is over the form.
<i>Enabled</i> input. If	Determines whether the form is able to receive user input. If Enabled is set to False, all controls contained by the form are likewise disabled.
<i>Font</i> by	Sets the default font for the form. All controls contained by the form will also adopt this font unless their <i>Font</i> property is set separately.
<i>ForeColor</i> used	Indicates the forecolor of the form, which is the color to display text. All controls contained by the form will also adopt this forecolor unless their <i>forecolor</i> property is set separately.
<i>FormBorderStyle</i> border	Indicates the appearance and behavior of the form and title bar.
<i>HelpButton</i>	Indicates whether the form has a Help button.
<i>Icon</i>	Indicates the icon that is used to represent this form.
<i>Location</i> property	When the <i>StartPosition</i> property is set to Manual, this indicates the starting location of the form relative to the upper left-hand corner of the screen.
<i>MaximizeBox</i>	Indicates whether the form has a maximize box.
<i>MaximumSize</i> property	Determines the maximum size for the form. If this is set to a size of (0,0), the form has no upper size limit.
<i>MinimizeBox</i>	Indicates whether the form has a minimize box.
<i>MinimumSize</i> resize	Determines the minimum size to which the user can resize the form.
<i>Opacity</i>	Represents the opacity, or conversely the transparency of the form from 0% to 100%. A form with 100% opacity is completely opaque, and a form with 0% opacity is completely transparent.
<i>Size</i>	Gets and sets the initial size of the form.
<i>StartPosition</i>	Indicates the position of the form when the form is first displayed.
<i>Text</i>	Determines the text caption of the form.

<i>TopMost</i> other	Indicates whether the form always appears above all forms that do not have this property set to <i>True</i> .
<i>Visible</i>	Determines whether the form is visible when running.
<i>WindowState</i>	Determines whether the form is minimized, maximized, or set to the size indicated by the <i>Size</i> property when first shown.

FormBorderStyle

- *None* The form has no border and has no minimize, maximize, help, or control boxes.
- *FixedSingle* The form has a single border and the user cannot resize it. It can have a minimize, maximize, help, or control box as determined by other properties.
- *Fixed3D* The form's border has a three-dimensional appearance and the user cannot resize it. It can have a minimize, maximize, help, or controlbox as determined by other properties.
- *FixedDialog* The form has a single border and the user cannot resize it. Additionally, it has no control box. It can have a minimize, maximize, or help box as determined by other properties.
- *Sizable* This is the default setting for a form. The user can resize it, and it can contain a minimize, maximize, or help box as determined by other properties.
- *FixedToolWindow* The form has a single border and the user cannot resize it. The window contains no boxes except the close box.

Algunas propiedades, como *Font* o *Size* son más complejas. Sus valores son instancias de una clase o estructura:

```
aForm.Size = New Size(300,200) (ancho y alto)
```

Se puede trabajar con ancho y alto por separado

```
aForm.Width = 300  
aForm.Height = 200
```

StartPosition

- *Manual* : la posición inicial la marca la propiedad *Location*
- *CenterScreen*: centrado en la pantalla
- *WindowsDefaultLocation* The form is positioned at the Windows default location

- and is set to the size determined by the *Size* property.
- *WindowsDefaultBounds* The form is positioned at the Windows default location
- and the size is determined by the Windows default size.
- *CenterParent* The form's starting position is set to the center of the parent form.

Opacity and Transparency in Forms

Se usa para crear efectos visuales en el formulario. Varía de 0% a 100%. Una opacidad del 100% hace que el formulario sea sólido y visible. Una opacidad del 0% hace que sea completamente transparente.

```
aForm.Opacity = 0.5
```

```
For i = -1 To 1 Step 0.0005
```

```
Me.Opacity = System.Math.Abs(i)
```

```
Next
```

Button

Típico botón Aceptar de todas las aplicaciones windows, suele utilizarse para entrada de datos con el ratón.

Propiedades

- Name: nombre que define al objeto durante la ejecución del programa, es de diseño.
- AllowDrop: para permitir operaciones de arrastrar y soltar
- Anchor : anclaje, decidimos respecto de que márgenes se ancla, si se mueve el formulario respecto de esos márgenes se adapta al nuevo tamaño.
- Backcolor: color de fondo, pero no del botón que no cambia, sino de un pequeño cuadro que lo rodea, no se aprecia.
- BackgroundImage: Imagen de fondo
- CausesValidation: controla si cuando un botón pierde el foco se activa o no el evento validate.
- ContextMenu : menú contextual
- Cursor :
- DialogResult:
- Dock: “Acoplamiento”, fijación de un control en los bordes del padre, si se redimensiona éste también lo hace el control. Para determinar esos márgenes ponemos la propiedad DockPadding del padre
- Enabled: para habilitar o deshabilitar el botón, deshabilitado no tiene efecto.
- FlatStyle: tipos de botón
- Font: tipo de fuente
- ForeColor: color del primer plano
- Image: imagen que ponemos en primer plano
- ImageAlign: posición donde la colocamos
- ImageIndex: más adelante
- ImageList:
- Location: posición del control
- Locked:
- Modifiers: ámbito del control
- RightToLeft:
- Size
- TabIndex: nº de orden para el tabulador, cuando tenemos varios botones podemos desplazarnos por ellos con el tabulador, el orden en el que reciben el foco depende del índice que tengan en esta propiedad.
- TabStop: si está a false el botón no toma el foco al pulsar el tabulador, aunque mantiene su propiedad anterior.
- Tag: para asociar alguna información adicional al botón
- Text:
- TextAlign

- Visible: para hacer invisible un botón, no recibe eventos

LABEL

Control que nos permite presentar un texto, este no puede cambiarse por el usuario.

Propiedades

- Name: nombre que define al objeto durante la ejecución del programa, es de diseño.
- AllowDrop: para permitir operaciones de arrastrar y soltar
- Anchor : anclaje, decidimos respecto de que márgenes se ancla, si se mueve el formulario respecto de esos márgenes se adapta al nuevo tamaño.
- AutoSize: si el tamaño de la etiqueta se ajusta al texto que contiene, siempre y cuando no aliniemos el texto (TextAlign)
- Backcolor: color de fondo, pero no del botón que no cambia, sino de un pequeño cuadro que lo rodea, no se aprecia.
- BorderStyle: tipo de borde
- CausesValidation: controla si cuando un botón pierde el foco se activa o no el evento validate.
- ContextMenu : menú contextual
- Cursor :
- Dock: “Acoplamiento”, fijación de un control en los bordes del padre, si se redimensiona éste también lo hace el control. Para determinar esos márgenes ponemos la propiedad DockPadding del padre
- Enabled: para habilitar o deshabilitar el botón, deshabilitado no tiene efecto.
- FlatStyle: tipos de botón
- Font: tipo de fuente
- ForeColor: color del primer plano
- Image: imagen que ponemos en primer plano
- ImageAlign: posición donde la colocamos
- ImageIndex: más adelante
- ImageList:
- Location: posición del control
- Locked:
- Modifiers: ámbito del control
- RightToLeft:
- Size
- TabIndex: nº de orden para el tabulador, cuando tenemos varios botones podemos desplazarnos por ellos con el tabulador, el orden en el que reciben el foco depende del índice que tengan en esta propiedad.
- Tag: para asociar alguna información adicional al botón

- Text:
- TextAlign
- UseMnemonic: se activa la etiqueta pulsando la letra a la que anteponeamos &
- Visible: para hacer invisible un botón, no recibe eventos

TEXTBOX

Cajas de texto, se utilizan para presentar textos y permitir al usuario que los modifique o para que introduzca él la información.

Propiedades

- Name:
- AcceptsReturn: indica si al presionar INTRO en la caja, se crea una nueva línea en cajas multilínea o se activa el botón predeterminado del form.
- AcceptsTab: indica si se registra el tabulador en la caja o se pasa el foco al sgte control del form.
- Accesibilidad:
- AllowDrop
- Anchor
- AutoSize: se ajusta a su contenido, solo en cajas de una línea
- Backcolor:
- Borderstyle:
- CausesValidation. Controla si cuando un objeto pierde el foco se activa su evento Validate.
- CharacterCasing: para conversiones automáticas de may a min,.....
- ContextMenu
- Cursor
- Dock
- Enabled:
- Font:
- Forecolor:
- HideSelection: si está a true cuando tenemos texto seleccionado y la caja pierde el foco, este no sigue resaltado. En caso contrario si está a false
- ImeMode
- Lines: contenido de una caja multilínea
- Location
- Locked: establece si el control se puede mover o cambiar de tamaño (en diseño)
- MaxLength: longitud máxima del texto
- Modifiers:
- Multiline: permite varias linea de texto

- PasswordChar: indicamos un carácter, y lo que escribimos en la caja no se ve, en su lugar aparece ese carácter. No tiene efecto si la anterior está a true
- ReadOnly: caja de sólo lectura
- ScrollBars: si la caja tiene multilíneas se pueden colocar barras de desplazamiento horizontales o verticales. Nos permite tener una caja de tamaño pequeño y poder leer toda la información
- Size: tamaño en píxeles
- TabIndex:
- TabStop
- Tag:
- Text
- TextAlign : alineación del texto en la caja
- Visible:
- WordWrap : si está a true en una caja multilínea cuando llegamos al final salta de línea, si está a false se desplaza el control en horizontal

Veamos otras propiedades de ejecución que afectan a como seleccionar texto y como se comporta esta selección

- SelectionLength: ancho del punto de inserción, se seleccionan ese número de caracteres a partir del punto de inserción, si escribimos cuando un texto está seleccionado se reemplaza. (Empezamos en 0)
- SelectionStart: si no se ha seleccionado texto, indica la posición del punto de inserción, si hay texto seleccionado indica la posición inicial de la selección. Valor 0 es la posición inicial a la izqda.
- SelectionText: devuelve o establece una cadena con el texto seleccionado actualmente. Si no hay selección "".

Eventos

- Keypress: asociado a otros muchos controles, se produce cuando el objeto recibe una tecla mientras tiene el foco. Recibe un argumento de tipo KeyPressEventArgs que contiene los datos relacionados con el evento:

Handled: obtiene o establece un valor que controla el evento

KeyChar: código ascii de la tecla pulsada (solo caracteres)

Ejemplos:

```
If !(Char.IsDigit(e.KeyChar) || Char.IsControl(e.KeyChar) )  
    e.handle = true
```

```
if ( Convert.ToInt16(e.KeyChar)==13)  
    //has pulsado INTRO
```

- KeyDown y KeyUp : eventos análogos al anterior, se producen cuando se pulsa o suelta una tecla. Su segundo parámetro es un objeto KeyEventArgs con las siguientes propiedades:

Alt, Shif, Control : estado de las teclas de control

keyCode: la tecla que se está pulsado como un valor enumerado de Keys

keyvalue: la tecla como código numérico

handled: análoga a KeyPress

Ejemplo

```
If (e.shift && e.KeyCode==keys.F2
```

- Validating, Validated : otros eventos asociados a la validación, si la propiedad CausesValidation está a true cuando pasamos el foco a otro control que tenga esta propiedad a true se dispara el evento Validating. Este evento tiene la propiedad Cancel para controlar esa pérdida de foco (si la ponemos a true mantenemos el foco en la caja). Cuando se ha terminado la validación se produce automáticamente Validated.
- TextChange: se produce si el contenido del control(text) ha variado, como y cuando depende del control. Con este en concreto ocurre cuando cambiamos el contenido de la caja , es decir su propiedad text si lo hacemos desde código o lo hace el usuario sobre la caja. Este evento puede provocarse a si mismo.

Métodos

- Clear: limpia la propiedad text
- Focus: para asignarle el foco a un control en tiempo de ejecución desde código. El foco solo puede pasarse a controles que estén visibles (no en evento load)
- Cut , Copy : para aplicar a la selección actual y copiar o mover al portapapeles.
- Paste: reemplaza la selección actual del textbox por el contenido el clipboard.. Cuando se sale de la aplicación se borra el portapapeles.
- Undo: deshace la última operación de edición en la caja si es que esta puede deshacerse , para saber esto último se dispone de la propiedad CanUndo

MASKTEXTBOX

Es un control que permite definir un patrón para el texto que contiene.
La propiedad mask permite especificar el formato del texto.

RADIOBUTTON Y CHEKBOX

Botones de elección y opción. El segundo de ellos permite establecer o anular una opción haciendo click, cada casilla de verificación es independiente de las demás. Un RadioButton es parecido en cuanto a que el usuario puede activar o desactivar una opción, pero cada uno no es independiente del resto, solo puede estar uno activo, si necesito tener varios debo agruparlos en un marco.

Propiedades (Radio)

- AutoCheck:
- ChekAlign : posición de la casilla de verificación
- Checked: si está o no activada la casilla
- TextAlign: alineación del texto

Propiedades (CheckBox)

- Appearance
- CheckState
- ThreeState

CONTROLES CONTENEDORES

GROUPBOX

Es un marco, se suele utilizar para agrupar dentro otros controles, principalmente RadioButtons, para ello se traza primero el marco y después se meten los controles dentro, si lo hago al contrario aunque estén encima no pertenecen a él. Si el marco es invisible tb lo son los controles que hay en él. No puede llevar barras de desplazamiento

Tiene propiedades ya vistas para otros controles

PANEL

Análogo al GroupBox pero se puede ocultar su borde. Sí puede llevar barras de desplazamiento.

FLOWLAYOUTPANEL

Es una subclase del control Panel. Los controles que contiene se ajustan a su tamaño y se colocan según indica la propiedad FlowDirection. Si no queremos que esto ocurra pondremos a False la propiedad WrapContents

TABLE LAYOUT PANEL

Análogo al anterior. Genera una tabla en la que cada celda puede contener un control

TABCONTROL

Típicas pestañas de Windows que pueden contener cada una de ellas otros controles. Estas pestañas son TabPages. Cada una de ellas tiene sus propias propiedades. A través de la propiedad SelectedIndex sabemos que pestaña está activa. Para detectar el momento en que el usuario activa otra pestaña utilizaremos el evento SelectedIndexChanged.

FUNCIÓN MESSAGEBOX

MessageBox.show(prompt, título, botones, icono, botón por defecto, opciones de pantalla)

Todas las opciones se proporcionan a través de enumerados, en la última si elegimos `rightalign` el texto aparecerá a la derecha (ver el resto en la ayuda)

Nos retorna el botón que el usuario ha pulsado, es un enumerado de la clase `DialogResult`

CONTROLES DE LISTA

LISTBOX

Hasta ahora conocemos varias formas de solicitar información al usuario, pero en ocasiones puede ocurrir que este tenga que elegir una de varias opciones que no son conocidos durante el diseño (no podemos usar `checkbox`, `optionbutton`,). Esta es una de las utilidades del `listbox`.

A simple vista es como un `textbox`, pero sus posibilidades son más amplias.

Una lista es un control capaz de contener cadenas de texto y también puede contener una colección de objetos que aparecen en una ventana donde el usuario puede elegir una o varias de esas opciones. Si el nº de elementos supera el nº que puede mostrarse se agrega automáticamente una barra de desplazamiento.

Desde código puede conocerse que elemento está seleccionado, añadir o eliminar elementos,.....

Propiedades

1. En Tiempo de Diseño

- `ColumWidth` : ancho de las columnas cuando la propiedad `MultiColumn` está a `true`. Si la dejamos a 0, el propio control calcula el más adecuado.
- `DataSource`: indica la colección de objetos que este control utiliza para obtener sus elementos. Puede ser un `array`, `arraylist`,.....

- **DisplayMember:** si los elementos de este control van a ser objetos de una clase, con esta propiedad determino la propiedad de los objetos con los que voy a llenar la lista
- **DrawMode:** para determinar el aspecto de los elementos
- **FormatString:** para dar formato a los elementos del listbox. Por ejemplo, si tenemos una lista de valores monetarios..... Podemos optar entre los formatos predefinidos o decidir uno propio. Mirar en la ayuda las posibilidades
- **FormattingEnabled:** determina si se usa el formato indicado en la anterior propiedad. Si es true sí lo utilizamos
- **HorizontalExtent:** esta propiedad tiene sentido cuando dibujamos nosotros los elementos del ListBox
- **HorizontalScrollBars:** muestra barra cuando los elementos no se ven enteros
- **IntegralHeight:** si está a true el último elemento se muestra entero, si está a false despendiendo el tamaño del ListBox puede verse a medias
- **ItemHeight:** altura en píxeles de los elementos cuando estos son dibujados
- **Items:** elementos de la lista
- **MultiColum:** para tener varias columnas. Va rellenando con los elementos cuando no le caben más (según el alto del list) introduce nueva columna.
- **ScrollAlwaysVisible:** si queremos siempre visible la barra de desplazamiento aunque no la necesitemos por tamaño.
- **SelectionMode:** determina cuantos y como elementos se pueden seleccionar, uno o varios:

MultiSimple: podemos elegir varios pero de uno en uno

MultiExtended: podemos elegir varios y arrastrando la selección

- **Sorted:** ordena los elementos, si la tengo a true, ordeno y luego la pongo a false se quedan ordenados como estaban pero si introduzco nuevos valores, éstos ya no están ordenados.
- **ValueMember:** si los elementos son objetos de una clase, determina a que propiedad que no es la que aparece en el list tenemos acceso. Lo haremos de otra manera

Además de estas propiedades tenemos otras de ejecución para poder añadir, buscar, eliminar, seleccionar,... elementos de un ListBox.

En Tiempo de Ejecución

- La propiedad Ítems reúne a todos los elementos contenidos en el List, es una colección (objeto iCollection)

Items.Count: nº de elementos (de lectura, no de escritura)

Listbox1.items.count

Items[índice] : accede al elemento con ese índice, para leerlo o modificarlo

- Para trabajar con elementos seleccionados

SelectedIndex: nº de orden del elemento en la lista cuando éste está seleccionado. Se empieza en 0. Si no se tiene ningún elemento seleccionado está a -1. Sólo es válida si la lista es de selección única.

List1selecedindex= 3 "Selecciono desde código ese elemento"
If listbox1.selectedindex = -1 then

SelectedIndices: colección de índices seleccionados cuando el ListBox es de selección múltiple

Selectedindices.count: nº de elementos seleccionados

SelectedItem: obtiene el elemento seleccionado (todo el objeto)

SelectedItems: colección de elementos seleccionados

Métodos

- GetSelected(índice): nos retorna el estado de selección de ese elemento

If ListBox1.GetSelected(0) = true then

- Items

Add(elemento) : para añadir un elemento

Listbox1.Items.Add(string)

AddRange : para agregar elementos cuando son objetos de una clase o colección

Insert (índice, elemento) : inserta el elemento antes del que tiene ese índice

RemoveAt(índice) : para eliminar ese elemento

List1.Items.RemoveAt(1)

indexOf : nos devuelve la posición del elemento en la lista (-1 si no está)

contains: busca un elemento y nos devuelve un booleano si está o no.

Con estos dos métodos pasamos como parámetro un objeto al buscar

Clear: para eliminarlos todos

- FindString (string, índice de inicio): busca el primer elemento que comienza por esa cadena a partir de esa posición (este segundo parámetro es opcional), retorna la posición del mismo. Si no lo encuentra retorna -1
- FindStringExtract: la cadena buscada coinciden exactamente. Tiene los mismos parámetros, si el 2º no se pone se busca desde el principio.

Eventos

El evento SelectedIndexChanged se produce cuando cambia la selección de elementos, este último se produce también si el usuario está cambiando dicha selección al arrastrarla con las teclas de desplazamiento

Click no detecta la selección con teclas de desplazamiento

CONTROL COMBOBOX

Este control (lista combinada) es una combinación de un textbox y un listbox. En la parte del text muestra el elemento seleccionado y permite introducir uno nuevo y la parte del list se despliega para funcionar como una lista con los elementos actuales. Cuando tecleo un elemento en el text, tengo que introducirlo en el list, no se hace automáticamente.

Propiedades

Las de los controles que lo forman

- DropDownStyle: forma del combo, por defecto se muestra sólo el textbox y la lista no.
 - DropDown: la lista no aparece inicialmente desplegada
- Simple: la lista está siempre desplegada. Para que salga entera aumentar
- DropDownList: no deja introducir texto en la caja, aunque si muestra el elemento seleccionado.
- DropDownWidth: ancho del textbox
- MaxDropDownItems: nº máximo de elementos que muestra el List a la vez

En tiempo de ejecución

- SelectedText: texto que se selecciona en el text si el combo es editable.

CONTROL CHECKEDLISTBOX

Variedad del ListBox que muestra una casilla de verificación a la izquierda de cada elemento. Hereda todas las propiedades del anterior y tiene algunas propias. Podemos tener varias casillas activadas pero solo un elemento seleccionado.

Propiedades

- CheckOnClick : si se pone a true las casillas se activan o desactivan con un clic

- ThreeDCheckBoxes : para mostrar las casillas tridimensionales

Propiedades en ejecución

Se han sustituidos las colecciones SelectedItems y SelectedIndices por CheckedItems y CheckedIndices

FOCO

Ya vimos una forma de ir pasando el foco de un control a otro desde diseño (tabindex), tambien podemos trabajar con el foco desde ejecución:

Metodo focus : para asignarle el foco a un control o formulario

Objeto.focus()

El foco solo se puede mover a un formulario o control visible. Como un formulario y sus controles no son visibles hasta que el evento Load termina, no se puede usar este método en este evento a menos que antes se utilice el método show para mostrar el formulario antes de que Load termine.

Tampoco se le puede pasar el foco a un objeto cuya propiedad enabled es false

Evento Gotfocus: ocurre cuando el objeto recibe el foco (click , tabulador , setfocus). Suele usarse para especificar acciones que ocurren cuando un objeto recibe el foco (p.ej. mostrando mensajes breves o instrucciones en la barra de estado)

Evento LostFocus: ocurre cuando el objeto pierde el foco.

Ejemplo:

```
Private text1_Gotfocus(){
    Text1.backcolor= RGB(255,0,0)
    Label1.text ="text1 tiene el foco"
}

Private text1_lostfocus(){
    Text1.backcolor=RGB(0,0,255)
    Label1.text="text1 no tiene el foco"
```

```
}
```

EVENTOS KEYDOWN Y KEYUP

KeyDown: ocurre cuando el usuario presiona una tecla. Su segundo parámetro es un objeto `KeyEventArgs` con las siguientes propiedades:

- `Alt`, `Shift`, `Control` : estado de las teclas de control
- `keyCode`: la tecla que se está pulsado como un valor enumerado de `Keys`
- `keyvalue`: la tecla como código numérico
- `handled`: análoga a `KeyPress`

Ejemplo

```
Private TxtFecha_KeyDown (....., e as KeyEventArgs .....){  
    If e.shift and (e.KeyCode=keys.F2)  
        Txtdate.text=today.date  
}
```

KeyUp: análogo al anterior para cuando soltamos la tecla

EVENTOS DE RATÓN

Tenemos 4 sucesos estándar: `MouseMove`, `MouseDown`, `MouseUp`, `MouseWheel` para cuando nos mueven el ratón encima de un control, nos pulsamos o nos sueltan o para cuando movemos la rueda. Reciben un parámetro `MouseEventArgs` con las siguientes propiedades:

`Button`: nos dice el botón del ratón que está pulsado

`X,Y`: posición del ratón

`Clicks`: nº de clicks que se han producido desde el último evento

Además todos los controles tienen los eventos `MouseHover` (análogo a `MouseMove`), `MouseEnter` y `MouseExit` se disparan cuando el ratón entra o sale del área del control.

MENÚS Y BARRAS

Hay dos tipos de menús : MenuStrip y ContextMenuStrip

Menustrip

Se añade el control y sobre el propio formulario se van escribiendo los menús. Es conveniente cambiarles el nombre, si no los coloca automáticamente y luego es difícil reconocerlos.

Para acceder a las propiedades: propiedad Ítems (colección de submenús que vamos a tener ToolStripMenuItem). Cada Submenú tiene a su vez una colección de DropdownItems, los menús contenidos cuando lo desplegamos . Puedo seleccionar el tipo : caja, combobox, separador u otro menú. Cada elemento menuItem tiene a su disposición la ventana de propiedades

ContextMenu

Se añade este objeto y la diferencia en cuanto al diseño del menú con respecto al anterior es que éste tiene un nodo del cual parte todo el resto de elementos que se llama ContextMenu. Por lo demás las propiedades son iguales que para el anterior.

Una vez diseñado el menú hay que asociarlo al control(les) que los van a mostrar, podemos hacerlo en diseño o ejecución

Textbox1.Contextmenu = contextmenu1

ToolStrip

Típica barra de herramientas de Windows. Su diseño es parecido al del MenúStrip. Puedo añadirle:

- Buttons
- Label : funcionan como un label o como un linklabel si la propiedad isLink está a true.
- DropDownButton: permite crear un menu que se despliega cuando se hace click en el botón
- SplitButton: combinación entre un button y StripDropDown. Tiene el evento ButtonClick y el evento Click.
- ComboBox
- TextBox
- Separator

StatusStrip

Barra de estado de diseño similar a los anteriores

CONTROLES PARA MANEJO DE FECHAS Y TIEMPO

TIMER

Permite establecer temporizaciones. No aparece en el formulario durante la ejecución, solo aparece en diseño

Propiedades

Interval: intervalo de tiempo en milisegundos en que se produce el evento Tick asociado al control y por tanto se ejecuta su código. Se puede variar en diseño o ejecución

Medio segundo = 500 milisegundos

Enabled: si se deshabilita el control, aunque interval $\neq$ 0 no se ejecuta el evento. Se puede variar en ejecución

Evento

Tick: se produce cada vez que se cumple un intervalo completo

Métodos

Start y Stop: para activar o desactivar el cronómetro

DATETIMEPICKET

Permite al usuario trabajar con fechas y horas usando una interfaz gráfica. Este control es similar a un comboBox. En su propiedad value tendremos la fecha elegida o la hora escrita en su text.

Cuando la propiedad Format tiene valor Long, sólo la fecha puede elegirse al desplegar el combo. Si está a time, el usuario puede escribir un nuevo valor en el text.

MONTHCALENDAR

Muestra sólo la parte del calendario. Tiene más propiedades que el anterior

- *AnnuallyBoldedDates*: Contiene un array de fechas que aparecerán en negrita cada año
- *BoldedDates*: Contiene un array de fechas que aparecerán en negrita
- *FirstDayOfWeek*: Día de la semana en el que empezamos
- *MaxSelectionCount* : máximo de fechas que se pueden seleccionar, para seleccionar una haga click, para seleccionar un rango pulso Shift. La cantidad de fechas en dicho rango no puede exceder el valor de esta propiedad.

Para acceder a las fecha seleccionadas tenemos la propiedad SelectedRange que consta de Start (primera fecha) y End (segunda fecha)

CONTROLES DE IMÁGENES

PICTUREBOX

Para contener imágenes de distinto formato: bmp, jpg, gif, icon. Las imágenes pueden ser un recurso de la aplicación o pueden cargarse desde fuera de ella. Si es un recurso de la aplicación utilizamos la propiedad Image, si se carga desde una dirección Web o disco ImageLocation.

Propiedades

- *ErrorImage*: imagen que se cargará si las anteriores propiedades fallan
- *SizeMode* para controlar si la imagen se ajusta al tamaño del control o viceversa.

En cuanto a la propiedad Cursor, se puede crear uno en ejecución:

```
PictureBox1.Cursor= new Cursor("ruta");
```

Se puede cargar una imagen en tiempo de ejecución

```
PictureBox1.Image= Image.FromFile ("ruta")
```

CONTROL IMAGELIST

Control que utilizamos para contener una colección de imágenes (objetos image). En si mismo no tiene luego representación en las ventanas, lo utilizan otros controles para obtener de él sus imágenes (barra de herramientas). Crear la lista de imágenes en diseño es fácil a través de la propiedad Images

Propiedades

[ColorDepth](#)

Número de colores que se utilizan en las imágenes (8,16,24,32 bits)

[Images](#)

Obtiene [ImageList.ImageCollection](#) para esta lista de imágenes.

[ImageSize](#)

Obtiene o establece el tamaño de las imágenes de la lista de imágenes.

[ImageStream](#)

Obtiene el identificador del objeto [ImageListStreamer](#) asociado a esta lista de imágenes.

[TransparentColor](#)

Obtiene o establece el color que se va a tratar como transparente.

NUMERICUPDOWN CONTROL

Permite mostrar un rango de números y el usuario puede clikear para aumentar o decrementar el numero seleccionado.

Propiedades

- *Hexadecimal*: Indica si el valor numeric se muestra en hexadecimal
- *Increment* : El valor que aumentamos o disminuimos con cada click
- *Maximum*: Valor máximo del control
- *Minimum*: Valor mínimo del control
- *ThousandsSeparator* Indicates whether the culture-appropriate thousands separator
- will be used when displaying values greater than 1000
- *Value* : valor actual del control

DOMAINUPDOWN CONTROL

Similar al anterior. En lugar de mostrar un rango de números permite trabajar con una colección de string

Propiedades

- *Items* : colección de string que se muestran
- *ReadOnly* : indica si el usuario puede alterar el texto del control, escribiendo texto en lugar de seleccionar un string de la colección. En este caso dicho texto no se añade a la colección de items
- *Text* : permite leer o escribir el texto en el control
- *Items* collection contains the strings that are displayed in the *DomainUpDown* control.

PROGRESSBAR CONTROL

Típica barra de progreso en la que fijamos un valor mínimo y un valor máximo.

La propiedad *Step* determina como aumenta el valor del control cada vez que ejecutamos el método *PerformStep()*.

Si la propiedad *Style* tiene valor *bloccs*, el control aumenta su valor con el método anterior. Puedo conseguir esto mismo si aumento la propiedad *value*

Si su valor es *marquee*, el control va cambiando de valor según el tiempo que marca la propiedad *MarqueeAnimationSpeed*

CERRAR UN FORMULARIO

Para cerrar un formulario utilizamos el método Close. Siempre que un formulario se está cerrando desencadena el evento FormClosing (antes de llegar a cerrarse). Este evento tiene un parámetro que nos permite cancelar dicha ejecución (e.cancel)

Formularios MDI

Una aplicación de tipo o estilo MDI (Multiple Document Interface), Interfaz de Documento Múltiple, se compone de un formulario principal, también denominado formulario MDI, que actuará como contenedor de otros formularios (documentos) abiertos durante el transcurso del programa, denominados formularios hijos o secundarios MDI. Como ejemplos de este tipo de aplicación tenemos PowerPoint o Access.

Dada su orientación de formulario contenedor, se recomienda limitar los controles en un MDI a los estrictamente necesarios. El menú es el ejemplo más identificativo de control idóneo para un formulario MDI, ya que a través de sus opciones, podremos abrir los formularios hijos de la aplicación.

Seguidamente describiremos el proceso de creación de un proyecto que contenga un formulario MDI y dos formularios hijos, así como el comportamiento de estos últimos cuando son abiertos dentro del formulario padre MDI.

- Para conseguir que este formulario tenga el comportamiento de un contenedor MDI, debemos asignar el valor True a su propiedad IsMdiContainer. También debemos establecer a este formulario como inicial en las propiedades del proyecto.
- En Cuanto a los formularios hijos, debemos instanciarlos(si queremos tener varios formularios del mismo tipo de hijo) para poderlos mostrar y para que se comporten como hijos debemos asignar a su propiedad MdiParent, la instancia actual del formulario en ejecución, es decir, this.

En muchas de estas aplicaciones en el formulario padre aparece una opción de menú Ventana que nos muestra los nombres de los formularios hijos abiertos, permitiéndonos cambiar de formulario activo al seleccionar una de esas opciones. Para conseguir esto establecemos en la propiedad MdiWindowListItem del MenuStrip, cual de sus submenús mostrará las ventanas.

Adicionalmente, y para darle un aspecto más profesional a este menú, podemos añadir los MenuItem correspondientes a la organización de los formularios hijos en Cascada, Mosaico Horizontal, etc. Para organizar los formularios abiertos en la aplicación en alguno de estos modos, deberemos ejecutar el método LayoutMdi() del formulario

MDI, pasándole como parámetro uno de los valores correspondiente a la enumeración `MdiLayout`.

Para acceder desde un formulario hijo a su MDI contenedor, disponemos de la propiedad `MdiParent`, que nos devuelve una referencia de dicho formulario padre.

La clase `Form` tiene la propiedad `MdiChildren`, que devuelve un array con todos los formularios hijos abiertos hasta el momento. Esto nos permite recorrer todo este conjunto de formularios para realizar operaciones con alguno de ellos o todos.